

KahvilaKoodi

Paperinen työpaketti

Viikot 34–49

Tämä paketti on aikataulu ja tarkistuslista tilanteisiin, joissa sivusto ei ole auki. Projektipäiväkirja kirjoitetaan sivustolla ja viedään repositoryn project-docs-kansioon. Rasti tässä vihossa ei ole palautus: työ on aina Git-repositoryssa.

Aikataulu

Vko	Pvm	Viikon aihe	Vaihe
34	17.–21.8.	Aloitukset: peli-idea ja työkalut	A
35	24.–28.8.	Pelin suunnitelma: ruudut ja feature-lista	A
36	31.8.–4.9.	Feature 1 · Ensimmäinen pelattava kierros	A
37	7.–11.9.	Feature 2 · Kahvilan oikea tuotelista	A
38	14.–18.9.	Feature 3 · Kello, pisteet ja palaute	B
39	21.–25.9.	Feature 4 · Kasvava kiire	B
40	28.9.–2.10.	Feature 5 · Top 5 -tuloslista	B
41	5.–9.10.	Katselmointi · Asiakas pelaa peliäsi	B
42	12.–16.10.	Syysloma – ei projektityötä	–
43	19.–23.10.	Feature 6 · Asiakkaan toivoma parannus	C
44	26.–30.10.	Feature 7 · Peli ohjaa pelaajaa itse	C
45	2.–6.11.	Pelin laatu · Peli kestää pelaamista	C
46	9.–13.11.	Pelin laatu · Koodi, jonka toinen ymmärtää	C
47	16.–20.11.	Pelin julkaisu · Ensimmäinen julkaisuehdokas RC1 testipelattavana	D
48	23.–27.11.	Pelin julkaisu · v1.0 julki	D
49	30.11.–4.12.	Näyttö · Aineisto ja luovutus	D

Vaihe A – Pelin ydin: idea, ensimmäinen pelattava kierros ja oikea tuotelista

Vko 34 · 17.–21.8. – Aloitus: peli-idea ja työkalut

Viikon jälkeen tiedät, millainen peli tehdään ja kenelle. Tyhjä peli käynnistyy selaimessa.

- ☐ Lue toimeksianto, laadi vähintään 8 kysymystä ja pidä aloituskeskustelu.
- ☐ Asenna sovittu Unity-versio Unity Hubissa, luo 2D-projekti ja varmista toimiva WebGL-testibuild.
- ☐ Perusta Git-repository, README, .gitignore ja project-docs-kansio.

Valmis kun: Asiakkaan vastaukset, avoimet asiat ja oletukset on eroteltu. Toinen henkilö löytää repositoryn README:stä projektin tavoitteen, ja WebGL-testibuild käynnistyy.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: asiakastarpeen muistiinpano, repository-linkki, ensimmäinen commit ja testibuild.

Vko 35 · 24.–28.8. – Pelin suunnitelma: ruudut ja feature-lista

Viikon jälkeen peli on paperilla: kolme ruutua ja featuret tekojärjestyksessä. Asiakas on hyväksynyt rajauksen.

- ☐ Täytä GDD:n omat päätökset sivustolla (mm. tavoite ja oma rooli omin sanoin) ja vie ladattu gdd.md repositoryn project-docs-kansioon.
- ☐ Tee priorisoitu tehtävälista eli backlog (P0, P1 ja P2), työmääräarviot ja valmis kun -ehdot.
- ☐ Piirrä käyttöliittymäluonnos (mockup) ja suunnittele Unityn scene, Canvas-paneelit, C#-skriptit, JSON ja tallennus.

Valmis kun: Pakollinen perusversio on hyväksytty. Jokaisella P0-tehtävällä on 0,5–1 päivän arvio ja havaittava valmis kun -ehto. Mockupissa näkyvät valikko, peli ja tulos.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: hyväksytty raja, päivätty mockup, tekninen suunnitelma ja backlog.

Vko 36 · 31.8.–4.9. – Feature 1 · Ensimmäinen pelattava kierros

Peliä voi pelata ensimmäistä kertaa: Aloita → asiakas tilaa kahvin → toimitat → piste → tulosruutu.

- ☐ Rakenna pelin kolme ruutua: tee Unityn Canvasille MenuPanel, GamePanel ja ResultPanel väliaikaisella grafiikalla.
- ☐ Toteuta yksi asiakas, yksi tilaus, oikea toimitus ja piste.
- ☐ Kirjaa ensimmäiset testit ja bugit.

Valmis kun: Aloita → Kahvi → Toimita → pisteet → aika loppuu → tulos toimii WebGL-buildissa ilman, että muutat Unity Editorissa objekteja kesken pelin.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: pelattava polku, video tai kuva ja vähintään kaksi testimerkintää.

Vko 37 · 7.–11.9. – Feature 2 · Kahvilan oikea tuotelista

Tilaukset arvotaan tuotelistasta. Valikoimaa voi muuttaa koskematta koodiin.

- ☐ Anna kahvilalle oikea tuotelista: luo products.json ja lataa se Unityn TextAsset-tiedostoviitteenä C#-olioiksi JsonUtilitylla.
- ☐ Toteuta 1–3 tuotteen tilaukset ja oikean/väärän toimituksen tarkistus.
- ☐ Pidä toimintalogiikka erillään käyttöliittymästä ja dokumentoi vastuut.

Valmis kun: Kahvin pistearvon muuttaminen JSONissa näkyy pelissä ilman C#-muutosta. Puuttuva tai rikkinäinen JSON näyttää hallitun virheen eikä riko koko peliä.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: JSON-esimerkki, latauslogiikan commit sekä testit tyhjälle ja virheelliselle datalle.

Vaihe B – Pelin featuret: kiire, kasvava vaikeus, top 5 ja asiakaskatselmointi

Vko 38 · 14.–18.9. – Feature 3 · Kello, pisteet ja palaute

Peliin tulevat kello, pisteet ja välitön palaute toimituksesta.

- ☐ Tuo kahvilaan kiire: toteuta C#:lla pisteytys, Time.deltaTime-ajastin ja pelin päättymisehdot.
- ☐ Näytä tilaus, aika, pisteet sekä onnistumisen ja virheen palaute.
- ☐ Testaa aika 0, nopea kaksoisklikkaus ja väärä tuote.

Valmis kun: Peli päättyy kerran ajan loppuessa, nopea kaksoispainallus ei anna kahta tulosta ja uusi peli nollaa ajan sekä pisteet.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: toimiva pelisilmukka, kolme testitapausta ja dokumentoitu bugikorjaus.

Vko 39 · 21.–25.9. – Feature 4 · Kasvava kiire

Peli vaikeutuu pistemäärän mukaan: isommat tilaukset ja lyhyemmät ajat, ei äkkihyppyjä.

- ☐ Toteuta sovittu vaikeuden kasvu ja pidä arvot helposti säädettävänä.
- ☐ Tee selkeät tilat: valikko, peli, tarvittaessa tauko ja tulos.
- ☐ Vertaile toisen henkilön kanssa kahta toteutusvaihtoehtoa ja kirjaa päätös.

Valmis kun: Valittu malli on perusteltu. Sovitut pistemäärät tuottavat joka kerta oikean tilaukseen ja asiakasajan ilman uutta if-ketjua jokaiselle tasolle.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: vaihtoehtomuistio, vaikeuslogiikan commit ja hyväksytyt testit.

Vko 40 · 28.9.–2.10. – Feature 5 · Top 5 -tuloslista

Viisi parasta tulosta ja nimimerkit säilyvät, vaikka pelin sulkee ja avaa uudelleen.

- ☐ Tee peliin top 5 -lista: tallenna ja lataa Unityn pienellä PlayerPrefs-avain-arvo-tallennuksella viisi parasta tulosta ja nimimerkkiä.
- ☐ Perustele tiedostomuoto ja tallennuspaikka teknisessä suunnitelmassa.
- ☐ Validoi luettu data ja nimimerkki; älä tallenna salaisuuksia.

Valmis kun: Kuudesta tuloksesta näkyy vain viisi parasta vielä sivun uudelleenavauksen jälkeen. Tyhjä tallennus, tasapisteet ja liian pitkä nimimerkki on testattu.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: tallennus uudelleenkäynnistyksen jälkeen sekä kirjatut riskit ja ratkaisut.

Vko 41 · 5.–9.10. – Katselmointi · Asiakas pelaa peliäsi

Asiakas pelaa peliä ensimmäistä kertaa ja antaa palautteen. Yksi muutos sovitaan.

- ☐ Tee asiakkaalle kokeiltava build ja valmiste 5–10 minuutin demo.
- ☐ Kerää palaute, selitä ratkaisut ymmärrettävästi ja sovi yksi rajattu muutos.
- ☐ Päivitä backlog, prioriteetit, arviot ja katselmointiloki.

Valmis kun: Asiakas on pelannut buildin alusta loppuun. Muistiossa näkyvät alkuperäinen palaute, oma tulkinta, päätös, hyväksyjä ja yksi rajattu issue.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: katselmointiloki, palaute, päätetty muutos ja toimiva build.

Vko 42 · 12.–16.10. – Syysloma

Ei projektityötä eikä korvaavia tehtäviä. Jatka viikolla 43 viimeisimmästä toimivasta main-versiosta.

Vaihe C – Peli valmiiksi: palautemuutos, käytettävyys, testaus ja koodin laatu

Vko 43 · 19.–23.10. – Feature 6 · Asiakkaan toivoma parannus

Asiakkaan pyytämä muutos on pelattavana. Vanha toiminnallisuus toimii edelleen.

- ☐ Muuta asiakkaan palaute pieniksi GitHub-issueiksi ja arvioi työmäärä.
- ☐ Toteuta sovittu muutos omassa feature-branchissa eli muutokselle rajatussa Git-haarassa.
- ☐ Katselmoi, testaa ja yhdistä muutos toimivaan main-versioon.

Valmis kun: Muutos täyttää hyväksymisehdon, vanha pelipolku toimii, katselmointiin on vastattu ja muutos on yhdistetty main-haaraan.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: GitHub-issue, oma haara ja yhdistäminen (merge) tai pull request, testit ja katselmointiloki.

Vko 44 · 26.–30.10. – Feature 7 · Peli ohjaa pelaajaa itse

Uusi pelaaja ymmärtää tavoitteen ja pelaa kierroksen ilman, että kukaan neuvoa vieressä.

- ☐ Vertaa toteutusta mockupiin ja kirjaa perustellut poikkeamat.
- ☐ Paranna luettavuutta, palautetta, ohjausta ja virhetilanteita.
- ☐ Pyydä vertaiselta lyhyt käytettävyystesti.

Valmis kun: Toinen käyttäjä löytää tavoitteen ja pelaa yhden tilauksen loppuun ilman suullista ohjetta. Kahdelle muutokselle näkyy ennen- ja jälkeen-tilanne.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: ennen/jälkeen-kuvat, vertaispalaute, päätökset ja korjaukset.

Vko 45 · 2.–6.11. – Pelin laatu · Peli kestää pelaamista

Peli ei kaadu rajatapauksissa: aika nollaan, tuplaklikit ja rikottu tuotelista on testattu.

- ☐ Laadi vähintään 12 testitapausta: normaali käyttö, rajatapaukset ja tallennus.
- ☐ Kirjaa kaikki aidosti löytyneet bugit. Osoita kolme täydellistä virheenkorjausketjua; käytä tarvittaessa opettajan erikseen merkitsemiä vikatehtäviä.
- ☐ Testaa puhdas käynnistys sekä puuttuva tai virheellinen datatiedosto.

Valmis kun: Kaikissa 12 testitapauksessa näkyvät build, lähtötila, toiminta, odotus, havainto ja tulos. Kolmessa ketjussa näkyvät havainto, syy, korjauscommit ja onnistunut uusintatesti.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: testausloki, tulokset, bugikorjaukset ja uusintatestit.

Vko 46 · 9.–13.11. – Pelin laatu · Koodi, jonka toinen ymmärtää

Peli toimii kuten ennen. Koodi on selkeämpi, ja osaat selittää ratkaisut.

- ☐ Poista turha toisto, selkeytä nimet ja rajaa osien vastuut.
- ☐ Pyydä koodikatselmointi ohjaajalta tai vertaiselta ja kirjaa palaute.
- ☐ Varmista, että osaat selittää myös tekoälyn avulla syntyneet ratkaisut.

Valmis kun: Sama nimetty testi läpäisee ennen ja jälkeen refaktoroinnin. Katselmointikommenttiin on vastattu ja pystyt selittämään ratkaisun ilman tekoälyn vastausta.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: katselmointimuistio, refaktorointicommit ja rakenteen perustelu.

Vaihe D – Julkaisu ja näyttö

Vko 47 · 16.–20.11. – Pelin julkaisu · Ensimmäinen julkaisuehdokas RC1 testipelattavana

Koko peli on pelattavana täsmälleen siinä muodossa, jossa se julkaistaan. Uusia ominaisuuksia ei enää lisätä.

- ☐ Tee ensimmäinen julkaisuehdokas (release candidate, RC1) ja jäädytä uusien ominaisuuksien lisääminen.
- ☐ Anna asiakkaan ja vähintään yhden muun käyttäjän testata peli.
- ☐ Luokittele palaute: korjataan, tunnettu puute tai myöhempään versioon.

Valmis kun: RC1 on sidottu yhteen committiin. Asiakas ja toinen käyttäjä ovat testanneet sen, ja jokaisella havainnolla on vakavuus, toistettavuus sekä päätös.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: RC1-build, testipalautteet, päätökset ja viimeinen korjauslista.

Vko 48 · 23.–27.11. – Pelin julkaisu · v1.0 julki

Peli on julkaistu. Linkki toimii muillakin koneilla, ja asiakas voi kokeilla peliä itse.

- ☐ Korjaa vain julkaisemisen estävät ja selvästi tärkeät virheet.
- ☐ Tee puhdas Unity WebGL -build, v1.0-tagin ja julkaise build GitHub Pagesiin.
- ☐ Testaa julkaistu linkki toisella laitteella tai selaimella ja kirjoita käyttöohje.

Valmis kun: v1.0-tagin commit vastaa julkaistua versiota. Toinen henkilö avaa linkin toisella selaimella tai laitteella ja pelaa yhden kierroksen ohjeen avulla.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: toimiva julkaisulinkki, v1.0/tagi, käyttöohje ja tunnetut puutteet.

Vko 49 · 30.11.–4.12. – Näyttö · Aineisto ja luovutus

Peli, repository ja projektipäiväkirja todistavat osaamisesi ilman suullista selitystä.

- ☐ Tarkista kaikkien viikkojen projektipäiväkirja ja vie valmis tiedosto repositoryn project-docs-kansioon.
- ☐ Täytä osaamisvaatimusten näyttömatriisiin linkit tai työnäytteet.
- ☐ Harjoittele demo ja selitä koodi, bugi, tallennus, Git ja tekoälyn käyttö.
- ☐ Luovuta peli, repository, projektipäiväkirja ja näyttöaineisto viimeistään pe 4.12.2026.

Valmis kun: Jokaisella arviointikohdalla on avautuva täsmälinkki tai tunniste. Projektipäiväkirja ja AI-loki ovat repositoryssä, ja demo käyttää samaa jäädytettyä v1.0-versiota.

Työnäyte Git-repositoryyn ennen rastia: Työnäyte Git-repositoryyn ennen rastia: peli, repository, project-docs/projektipäiväkirja.md, näyttömatriisi, AI-loki ja demo.

Sanasto

Projektin tunnukset ja ammattitermit siinä järjestyksessä, jossa ne tulevat vastaan. Jokainen on selitetty myös sivustolla ensimmäisen käytön kohdalla.

repository vko 34	<i>Projektin koodivarasto Gitissä.</i> Repository on projektin tiedostojen ja koko muutoshistorian säilytyspaikka. Tässä projektissa se on GitHubissa, ja sieltä löytyvät peliprojekti, project-docs-kansio ja työnäytteet.
commit vko 34	<i>Versionhallintaan tallennettu muutos.</i> Commit on yksi versionhallintaan tallennettu muutostyökalu: mitä muutit ja miksi. Jokaisella commitilla on oma tunnistus, johon voit linkittää työnäytteenä.
build vko 34	<i>Pelin ajettava julkaisuversio.</i> Build on Unityn tekemä valmis versio pelistä. Sitä pelataan ilman Unity-editoria, ja juuri sitä asiakas kokeilee.
WebGL vko 34	<i>Unityn selainjulkaisu.</i> WebGL on Unityn julkaisumuoto, joka toimii selaimessa ilman asennusta. WebGL-build on siis selaimessa pelattava julkaisuversio pelistä – tämän projektin lopputuote.
P0 vko 34	<i>Pakollinen ydin.</i> P0 on se osa peliä, jonka on pakko valmistua: ilman sitä peliä ei voi luovuttaa asiakkaalle. Tee koko P0 valmiiksi ennen kuin aloitat lisäominaisuuksia.
P1 vko 35	<i>Tärkeä jatkosisältö.</i> P1 on ominaisuus, joka tehdään vasta kun koko P0 toimii. Se parantaa peliä, mutta peli on luovutettavissa myös ilman sitä.
P2 vko 35	<i>Valinnainen lisä.</i> P2 on ominaisuus, joka voidaan jättää kokonaan pois, jos aika loppuu. Merkitse P2:ksi kaikki, mistä voi luopua ilman että asiakkaan vaatimus jää täyttämättä.
GDD vko 35	<i>Game Design Document, pelin suunnitteludokumentti.</i> GDD kokoaa yhteen tiedostoon pelin konseptin, pelin kulun, omat suunnittelupäätökset, rajauksen ja avoimet asiat. Tässä projektissa se täytetään Suunnitelma-näkymässä ja tallennetaan repositoryyn nimellä gdd.md.
GitHub-issue vko 35	<i>Tehtävä, jota seurataan GitHubissa.</i> GitHub-issue on yksi tehtävä: otsikko, perustelu, rajattu muutos ja valmis kun -ehto. Tässä projektissa yhden issuen työmäärä on 0,5–1 työpäivää.
backlog vko 35	<i>Priorisoitu tehtävälista.</i> Backlog on projektin tehtävälista, jossa jokaisella tehtävällä on prioriteetti (P0, P1 tai P2), työmääräarvio ja valmis kun -ehto. Backlog päivitetään aina, kun asiakas päättää jotain uutta.
UI vko 35	<i>User interface, käyttöliittymä.</i> Käyttöliittymä on se osa peliä, jonka pelaaja näkee ja jota hän käyttää: painikkeet, tekstit ja paneelit. Unityssä ne rakennetaan Canvas-alueelle.
feature vko 36	<i>Pelin yksittäinen ominaisuus.</i> Feature on yksi pelaajalle näkyvä ominaisuus, esimerkiksi pistelasku tai viiden parhaan tuloksen lista. Tässä projektissa featuret tehdään yksi kerrallaan, yleensä yksi viikossa.
asset vko 36	<i>Peliin tuotava valmis tiedosto.</i> Asset on peliin tuotava valmis kuva-, ääni- tai fonttiedosto, esimerkiksi sprite eli hahmon tai esineen kuva. Kirjaa jokaisesta assetista lähde ja lisenssi.
JSON vko 37	<i>Tekstimuoto datalle.</i> JSON on yksinkertainen tekstimuoto, jossa tieto on nimi–arvo-pareina. Tässä projektissa kahvilan tuotteet ovat products.json-tiedostossa, joten valikoimaa voi muuttaa koskematta koodiin.
branch vko 43	<i>Git-haara.</i> Branch eli haara on rinnakkainen kehityslinja: teet muutoksen omassa haarassa, jolloin toimiva main-haara pysyy ehjänä. Valmis haara yhdistetään eli mergetään takaisin mainiin.
pull request vko 43	<i>PR, pyyntö yhdistää haara pääversioon.</i> Pull request eli PR on GitHubissa tehtävä pyyntö yhdistää oma haara main-haaraan. Se antaa katselmoinnille oman paikan ennen yhdistämistä; pienen muutoksen voi myös yhdistää suoraan ilman PR:ää.
T01 vko 45	<i>Testitapauksen tunnus.</i> Testitapaukset numeroidaan juoksevasti: T01 on ensimmäinen testitapaus, T02 toinen. Odotettu tulos kirjataan ennen ajoa, ja tunnuksella viitataan testiin päiväkirjassa ja näyttömatrisissa.
RC vko 47	<i>Release candidate, julkaisuehdokas.</i> Julkaisuehdokas on lähes valmis versio, joka testataan täsmälleen siinä muodossa, jossa se aiotaan julkaista. RC1 on ensimmäinen julkaisuehdokas, eikä siihen enää lisätä uusia ominaisuuksia.

tagi

vko 47

Versionhallintaan merkitty nimetty versio. Tagi on yhteen committiin kiinnitetty nimilappu, esimerkiksi RC1 tai v1.0. Tagin avulla löydät myöhemmin täsmälleen sen version, jonka asiakas testasi.

Näyttömatriisi – 32 osaamisvaatimusta

Rasti vasta, kun vaatimukselle on täsmällinen työnäyte: linkki, commit, kuva, testirivi tai muistio. Sama työnäyte voi kelvata useaan kohtaan.

Ohjelmointi

- **Kehitysympäristö** – Editorikuva, build ja repository.
- **Virheenkorjaus** – Kolme täydellistä ketjua aidoista havainnoista tai opettajan merkityistä vikatehtävistä.
- **Toimintojen testaus** – 12 testiä odotettuine ja toteutuneine tuloksineen.
- **Rakenteinen ohjelmointi** – Vastuutettu koodirakenne ja katselmointi.
- **Ylläpidettävä koodi** – Selkeät nimet, pienet metodit ja refaktorointicommit.
- **Käyttöliittymä** – Mockup sekä ennen/jälkeen-kuvat.
- **Toimintojen toteutus** – GitHub-issuet, valmis kun -ehdot ja commitit.
- **Tehtävistä sopiminen** – Tehtävälista ja kokousmuistio.
- **Ratkaisuvaihtoehdot** – Kahden toteutustavan vertailu ja päätös.
- **Ratkaisujen arviointi** – Koodi- tai peliversion katselmointiloki.
- **Oma toiminta tiimissä** – Itsearviointi ja seuraava kehitysaskel.

Ohjelmistokehittäjänä toimiminen

- **Asiakkaan tarpeet** – Kysymyslista ja aloitusmuistio.
- **Asiakaslähtöinen viestintä** – Ymmärrettävä yhteenveto vaihtoehtoista ja rajoista.
- **Version katselmointi** – Lokit viikoilta 41 ja 47.
- **Priorisointi** – Backlog P0-, P1- ja P2-tasoinen ennen ja jälkeen palautteen.
- **Tehtäviksi jakaminen** – 0,5–1 päivän issuet ja nimetyt tekijät.
- **Suunnittelu ja arviointi** – Työmääräarvio verrattuna toteumaan.
- **Toimintalogiikka** – Tilaus, toimitus, pisteet, aika, vaikeus ja pelitilat.
- **Tietovaraston valinta** – JSON- ja tallennusratkaisun perustelu.
- **Yhteys tietovarastoon** – Tuotetietojen luku sekä tuloksen tallennus ja lataus.
- **Rajapinnat ja tieto** – Tiedonkulun kuvaus ja sitä vastaava koodi.
- **Tietoturva** – Syötteiden, tallennuksen ja salaisuuksien riskit sekä ratkaisut.
- **Versionhallinta** – Jatkuva Git-historia ja toimiva main.
- **Osan liittäminen** – Testattu yhdistäminen (merge) tai pull request.
- **Tuotantojulkaisu** – Toimiva v1.0-julkaisulinkki.

Ohjelmiston toteuttaminen ohjelmistokomponenttikirjastolla

- **Kehittäisympäristön käyttöönotto** – Unity Hub, sovittu Unity-versio, 2D-projekti ja WebGL-testibuild.
- **Kirjaston mahdollisuudet ja rajoitteet** – Viikon 40 kirjaus tallennusratkaisun rajoituksista ja viikon 48 julkaisupäätökset tunnettuine puutteineen.
- **Kirjaston toiminnot ja työkalut** – Canvas-käyttöliittymä, Inspector-kytkennät, Play Mode ja Build Profiles committeina ja kuvina.
- **Ulkoiset komponentit** – TextMeshPro ja grafiikkapaketit tuotuina; lähteet ja lisenssit kirjattu GDD:hen tai README:hen.
- **Suunnittelu, toteutus ja testaus kirjastolla** – GDD:n omat päätökset ja backlog, featuret 1–7 committeina, testitapaukset T01–T12.
- **Julkaisu asiakkaan ympäristöön** – Unity WebGL -build GitHub Pagesissa; asiakas on kokeillut julkaistua linkkiä.
- **Ohjelmiston dokumentointi** – README, käyttöohje, gdd.md ja projektipäiväkirja project-docs-kansiossa.

Muista: sivuston rastit ja kentät tallentuvat vain selaimeen. Ne eivät siirry opettajalle eivätkä korvaa Gitissä olevaa työtä.